Onyx — The Kits: a developer's guide

Onyx · multi-process OS on Raspberry Pi 4

*How an Onyx program is built on the system's shared libraries — the **kits** — with a short example for each. Each kit has its reference, with every operation it exposes (documents 10 to 19, linked in the table below). Companion of the Developer Guide (the build, the application model, the ports) and of Kernel Internals.*

Table of contents

1. One kit per domain
2. Using a kit in a program
3. AppKit — what makes a program run
4. UIKit — the interface
5. SystemKit — talking to the system and the other programs
6. NetKit — the network
7. FileKit — files, folders, archives
8. ImageKit — pictures
9. AudioKit — sound
10. FontKit — fonts and text
11. PrinterKit — printing
12. GPIOKit — the 40-pin header
13. GameKit — the games of the card's ROMs
14. Adding to a kit, creating a kit
15. Quick reference

1. One kit per domain

An Onyx program does not talk to the kernel, and does not carry copies of common code. It is built on **kits**: shared libraries, one per domain, loaded once for the whole system and bound to each program that uses them.

Kit	Domain	On the card	Sources	Reference
AppKit	What makes a program run: the system's calls, strings, the console, `.ini` files, starting programs	`SD:/lib/appkit.so`	`user/Kits/appkit`	10 — AppKit
UIKit	The interface:	`SD:/lib/`	`user/Kits/`	11 — UIKit

Kit	Domain	On the card	Sources	Reference
	windows, widgets, dialogs, the theme, icons	`uikit.so`	`uikit`	
SystemKit	Talking to the system and the other programs: notifications, clipboard, trash, volume, wallpaper, file associations, the dock, the language and the time zone	`SD:/lib/systemkit.so`	`user/Kits/systemkit`	12 — SystemKit
NetKit	The network: HTTP, the FTP volumes	`SD:/lib/netkit.so`	`user/Kits/netkit`	13 — NetKit
FileKit	Files and folders, paths, compression, archives	`SD:/lib/filekit.so`	`user/Kits/filekit`	14 — FileKit
ImageKit	Pictures: reading, writing, resizing, adjusting	`SD:/lib/imagekit.so`	`user/Kits/imagekit`	15 — ImageKit
AudioKit	Sound: playing files, notes, synthesis, the sound output	`SD:/lib/audiokit.so`	`user/Kits/audiokit`	16 — AudioKit
FontKit	Fonts: FreeType, the font manager, anti-aliased text	`SD:/lib/fontkit.so`	`user/Kits/fontkit`	17 — FontKit
PrinterKit	Printing: the Print dialog, a job's pages	`SD:/lib/printerkit.so`	`user/Kits/printerkit`	18 — PrinterKit
GPIOKit	The Raspberry Pi's 40-pin header: pins,	`SD:/lib/gpiokit.so`	`user/Kits/gpiokit`	19 — GPIOKit

Kit	Domain	On the card	Sources	Reference
	PWM, edges, I2C, SPI; a simulated board			
GameKit	The games of the card's ROMs: the emulators' consoles, the watched folders, the ROMs, their pictures	`SD:/lib/gamekit.so`	`user/Kits/gamekit`	20 — GameKit

Three rules follow from this layout, and they hold for every new development:

- **Reusable code goes into the adequate kit** — not into a new shared header, not copied into an application. If no kit fits the domain, a new kit is created (§14).
- **A program draws on the kits as much as it can**, and never reaches the kernel itself. Only AppKit reads the kernel's table; so the kernel can change without any program being rebuilt. (One exception, decided by the user: **GPIOKit** calls the kernel's `gpio_ctl` entry itself — it ships with the kernel, as AppKit does: §12.)
- **What one program alone uses stays beside that program**, in its own folder.

A kit has **one header**, `<kit>/<kit>.h`, which **declares**; the code is in the library. The memory cost of a kit is paid once: its code is shared by every program that uses it.

2. Using a kit in a program

Including

`user/Kits` is on every include path. A source includes a kit's header by the kit's name, whatever its own folder:

```
#include "appkit/appkit.h"          // AppKit
#include "uikit/uikit.h"            // UIKit
#include "systemkit/systemkit.h"    // SystemKit
#include "netkit/netkit.h"          // NetKit
#include "filekit/filekit.h"        // FileKit
#include "imagekit/imagekit.h"      // ImageKit
#include "audiokit/audiokit.h"      // AudioKit
#include "fontkit/uikitface.h"      // FontKit
#include "printerkit/printerkit.h"  // PrinterKit
```

Linking

AppKit needs nothing: the kernel loads it and binds it to every program. Every other kit is linked through its **import archive**, a few bytes of stubs and a constructor that opens the library before `main`:

The program is	It links
C++ (an application)	`lib/<kit>.imp.a`
C (a console tool)	`lib/<kit>.imp_c.a`

An archive only brings what the program calls: listing a kit a program does not use costs nothing, and the library is not opened. The applications' rules in `user/Makefile` already list UIKit, FileKit, SystemKit and NetKit; a console tool names its kits in `user/BinUtils/Makefile`:

```
# a C tool that uses SystemKit
mytool.elf: KITLIBS = ../lib/systemkit.imp_c.a
mytool.elf: ../lib/systemkit.imp_c.a
```

Versions

A kit's table only grows: a function is never removed nor renamed, so a newer library always runs an older program. A program built against a table that grew needs at least that library: its package says so (`needs = systemkit >= 1.56` in `tools/pkg/packages.ini`), and the package manager installs the library first.

From BASIC

A BASIC program uses the kits too: `#import filekit`, then `FileKit.copy (a$, b$, 0, 0)` — every function of a kit that takes and returns numbers, strings and pointers, by its name, and its structures as TYPEs (`DIM e AS FileKit.zip_entry`) (docs/04 §13, *The system's kits*). BASIC knows a kit by its **description**, `SD:/lib/<kit>.bi`, made from the kit's `.abi` and its headers by `tools/kitbi/kitbi.py` when the kits are built: nothing in BASIC names a kit, so **a new kit is importable as soon as it is built** (docs/03, *BASIC and the kits*). What BASIC cannot call — C++ classes, structures passed by value, a structure with a union or a bit field — is left out of the description: a kit meant for BASIC too exposes plain C functions.

On a PC

The same sources build on a PC for the tests and the screenshots. There is no shared library there: the kits' headers bring their code inline, and UIKit's sources are compiled with the program (its window API relayed to the stand-in kernel's window manager: `user/Kits/uikit/port.cpp`). Nothing changes in the program's code.

3. AppKit — what makes a program run

`#include "appkit/appkit.h"` — nothing to link.

AppKit is the program's link to the system: files, processes, time, sockets, the events' pump… (the `kapi_*` calls, listed in the Developer Guide; the windows are UIKit's since 2026-10-08: `uk_win_*`, §4), plus the small services every program needs: strings without a C library, the console, a reader of `.ini` files, the starting of other programs.

A console tool: read a file, print to the console.

```
#include "appkit/appkit.h"

int main (void)
{
```

```
    char buf[256];
    void *f = kapi_open ("SD:/etc/autostart");
    if (f == 0) { ax_putln ("no such file"); return 1; }
    int n = kapi_read (f, buf, sizeof buf - 1);
    kapi_close (f);
    buf[n > 0 ? n : 0] = '\0';

    ax_putln ("SD:/etc/autostart says:");
    ax_putln (buf);
    return 0;
}
```

Settings from an `.ini` file in the program's own folder:

```
if (app_ini_load ("config.ini") >= 0)
{
    const char *name = app_ini_get ("window", "title", "Untitled");
    int width        = app_ini_get_int ("window", "width", 640);
    for (int i = 0; i < app_ini_count (); i++)        // or every entry, in the file's order
        ax_putln (app_ini_key (i));
}
```

Strings (no C library needed):

```
char path[96]; int n = 0;
ax_strcat (path, sizeof path, &n, "SD:/docs/");       // never overflows, always terminated
ax_strcat (path, sizeof path, &n, "notes.txt");
char num[12]; ax_itoa (42, num);
if (ax_streq (num, "42")) ax_putln (path);
```

Starting a program — an application by its name, or a file by whatever runs it (a `.bas` program by BASIC, a game by its emulator):

```
lx_launch ("tinypad", "SD:/docs/notes.txt");          // an app, with its arguments
lx_open ("SD:/games/tetris.gb", 0);                   // a file: by its runner
```

The volumes, USB sticks (kapi v93) — list them, eject a stick, format one:

```
struct kapi_volume v[16];
int n = kapi_vol_list (v, 16, KAPI_VOLS_ROOM);        // SD:, SD1:.., USB1:.., USB1P1:..,
RAM:
for (int i = 0; i < n && i < 16; i++)
    if ((v[i].flags & KAPI_VF_REMOVABLE) && v[i].state == KAPI_VST_MOUNTED)
        ax_putln (v[i].label);                        // a stick plugged in: "USB" + its
label

if (kapi_vol_eject ("USB1:", 0) == -KAPI_EBUSY)       // files open on it (they were synced)
    kapi_vol_eject ("USB1:", KAPI_EJECT_FORCE);       // ... after asking the user

struct kapi_format f = { KAPI_FMT_EXFAT, 0, 0, "PHOTOS" };
kapi_vol_format ("USB1:", &f);                        // erases it; SD: is always refused
```

4. UIKit — the interface

`#include "uikit/uikit.h"` — link `lib/uikit.imp.a` (the applications' rules do).

A window is a `Root`; widgets are added to it; `run ()` is the event loop. Everything is in the namespace `uikit`.

A window, a label, a button:

```
#include "appkit/appkit.h"
#include "uikit/uikit.h"
using namespace uikit;

static Label *g_label;
static int g_count;

static void onClick (Widget &)
{
    char text[32]; int n = 0;
    ax_strcat (text, sizeof text, &n, "clicked ");
    char num[12]; ax_itoa (++g_count, num);
    ax_strcat (text, sizeof text, &n, num);
    g_label->setText (text);                          // the widget redraws itself
}

int main (void)
{
    Root root (320, 120, "Hello");                    // a decorated window
    g_label = new Label (12, 12, 296, 20, "Press the button");
    root.addChild (g_label);
    root.addChild (new Button (12, 44, 100, 28, "Click me", onClick));
    root.run ();                                      // until the window is closed
    return 0;
}
```

Dialogs are modal calls:

```
if (uk_messagebox ("Delete", "Delete this file?", MB_YESNO))
{
    char path[256];
    if (uk_file_open (path, sizeof path, "SD:/docs"))  // the file chooser
        g_label->setText (path);
}

unsigned colour = 0x2060C0;
if (uk_color_dialog (&colour, "Pick a colour")) root.setBg (colour);
```

Tabs (`TabStrip`, `uikit/tabstrip.h`): a row of closable tabs over the program's own view — the strip shows them, the program shows the chosen one's content. Each tab carries a pointer for the program; a close is only **asked** (the cross, a middle click): the program decides, then removes it.

```
static TabStrip *g_tabs;
static void onTab (Widget &)   { show (g_tabs->data (g_tabs->selected)); }
static void onClose (Widget &) { int i = g_tabs->closing; Doc *d = (Doc *) g_tabs->data (i);
                                 if (doc_close (d)) g_tabs->remove (i); }    // (a question
first, if need be)
static void onNew (Widget &)   { g_tabs->select (g_tabs->add ("Untitled", new_doc ()), true);
}
...
g_tabs = new TabStrip (0, 0, root.width, 30, onTab);
g_tabs->onClose = onClose; g_tabs->onNew = onNew;
g_tabs->anchor = ANCHOR_LEFT | ANCHOR_TOP | ANCHOR_RIGHT;
root.addChild (g_tabs);
```

`setTitle (i, s)`, `setMark (i, on)` (a dot: something runs, something new), `selectNext (±1)` (the program's Ctrl+Tab), `activeFace` (the chosen tab opens onto the content's colour). The Terminal's tabs.

The theme: draw with the palette (`C_BG`, `C_TEXT`, `C_ACCENT`, `C_FACE`…), never with fixed colours, so that the program follows the user's theme.

Text in an owner-drawn widget (`uikit/text.h`): `uk_text_fit` cuts a line with "…", `uk_text_wrap` word-wraps a paragraph into lines of a width (Stickies' cards), `uk_text_over` writes straight on a see-through window — the wallpaper — blended by coverage, with a soft shadow or engraved (the agenda's and Stickies' headers). The tool bar's icons (`uikit/toolbar.h`, `WKT_*`) include `WKT_TRASH` and `WKT_PIN`:

```
int st[6], ln[6]; bool more;
int k = uk_text_wrap (text, len, 196, 6, st, ln, &more);   // up to 6 lines of 196 px
for (int i = 0; i < k; i++) { char l[256]; int n = ln[i] < 255 ? ln[i] : 255;
    memcpy (l, text + st[i], n); l[n] = 0; uk_text (canvas, 22, 33 + 17 * i, l, ink); }
uk_text_over (canvas, 38, 9, "Pinned notes", 0x00FAFCFF, 2, 1);   // bold, a soft shadow, on the wallpaper
```

Without C++ (`uikit/flat.h`, in `uikit/uikit.h`): the same window and widgets as handles, behind plain C functions — what a BASIC program calls (`#import UIKit`), what QBStudio's generated code is made of, and what a C program can use:

```
void *win = uk_window ("Hello", 320, 120, 0);
uk_label (win, 12, 12, 200, 20, "Your name:");
void *name = uk_textbox (win, 12, 36, 200, 24, "", 0);
uk_button (win, 220, 36, 80, 24, "OK", on_ok);          /* void on_ok (void *button) */
uk_window_run (win);                                     /* until the window is closed */
```

```
#import UIKit
win = UIKit.window("Hello", 320, 120, 0)
ok = UIKit.button(win, 220, 36, 80, 24, "OK", ADDRESSOF(Clicked))
DO WHILE UIKit.window_wait(win): LOOP
```

`uk_panel (window, x, y, w, h)` is an area with widgets of its own — every maker takes the window *or a panel* as what holds the widget —: a page shown in place of another (`uk_show`), moved elsewhere (`uk_set_parent`). QBStudio's user controls are panels.

A widget added to UIKit that BASIC should reach gets its functions there (a maker, and `uk_set_text` / `uk_get_value` … taught its kind).

An icon — any picture ImageKit reads (BMP today, PNG tomorrow), given as icons are drawn (magenta = see-through):

```
#include "uikit/bmp.h"

int w, h;
unsigned *px = ui::icon_load ("SD:/apps/tinypad.app/icon.bmp", &w, &h);
if (px) { /* draw it ... */ delete [] px; }
```

A code editor (`uikit/codeedit.h`, 2026-10-06 — QBStudio's, made a widget of the kit for every program that edits code: QBStudio, Turtle Quest): BASIC in a monospaced face with its colours, the line numbers, the indentation's guides, a line lit (`hiLine`: an error's, a debugger's current line), the lines with a problem marked, undo / redo, the clipboard, the completion, UTF-8 comments:

```
CodeEdit *ed = new CodeEdit (10, 10, 400, 300);
ed->mono = myMonoFace;                  // (0: the UI's face)
```

```
ed->isKeyword = is_basic_word;          // bool (const char *w, int n): the words coloured
and put in capitals
ed->setText ("FOR i = 1 TO 3\n  PRINT i\nNEXT\n");
ed->onChange = changed;                 // Action: the text changed
root.addChild (ed);
ed->hiLine = 2; ed->showLine (1);       // line 2 lit and shown (hiLine 1-based, showLine 0-
based)
ed->clearMarks (); ed->addMark (3);     // line 3 has a problem
```

The completion: set `complete` (`void (CodeEdit &, const char *object)`), which calls `ed.addCompletion (name, kind, detail)` for each item offered after `object.` (or Ctrl+Space: `object` empty).

A timer or polling: derive from `Root` and override `onTick ()` (called about 60 times a second).

Several windows (2026-10-07, kapi v94): one more window of the program is a `Root` made with `NewWindow`; the first window's `run ()` serves them all (their events, `onTick`, drawing), a click in one makes it `Root::current ()`, its close box calls `onClose ()` (by default `closeWindow ()`; closing the first window ends the program). Telegram has one a conversation (docs/03 §6 *Several windows*).

```
class ChatWin : public Root
{
public:
    ChatWin (const char *who) : Root (NewWindow (), -1, -1, 500, 400, who) { if (!winOpened
()) return; /* its widgets */ }
    void onClose () override { closeWindow (); gone = true; }   // (deleted later, by the
main window's onTick)
    bool gone = false;
};
ChatWin *w = new ChatWin ("Alice");
if (!w->winOpened ()) { delete w; /* one window only: show it beside */ }
```

An icon in the menu bar's status area (kapi v95): `uk_tray ("SD:/apps/myapp.app/icon.bmp", "My App - 3 new")`; a double click on it shows the program's first window again (even minimised) and calls the first `Root`'s `onTray (KAPI_TRAY_OPEN)`, a right click `onTray (KAPI_TRAY_MENU)`; `uk_tray_clear ()` takes it away.

The window API (`uikit/win.h`, 2026-10-08; in `uikit/uikit.h`, or included alone by a C program, which links `lib/uikit.imp_c.a`): the program's windows and what it asks of the graphics server, as plain C functions — `uk_win_create`, `uk_win_present`, `uk_win_on_key`, `uk_win_menu_set`, `uk_win_list`, `uk_win_tray_set`, `uk_win_fullscreen_begin`… (they were AppKit's `kapi_create_window`, `kapi_present`…: docs/03 §5.10.1 has the table). A `Root` calls them for you; a game that draws its own canvas, a full-screen program or a shell program (the dock, the menu bar) calls them itself. Each graphics server has its own UIKit (the desktop's speaks to Elegant), so the same binary runs on every one; `uk_win_server` says which.

```
#include "appkit/appkit.h"
#include "uikit/win.h"

static void on_key (unsigned long sender, int ev, gui_value v) { (void) sender; if (ev ==
GUI_EVENT_KEY && v == 27) kapi_exit (0); }

int main (void)
{
    unsigned *fb = uk_win_create (320, 200, "Plasma");     /* the canvas: 0x00RRGGBB, 320 a
row */
    if (fb == 0) return 1;
    uk_win_on_key (on_key);
```

```
    for (int t = 0; !kapi_should_exit (); t++)
    {
        for (int i = 0; i < 320 * 200; i++) fb[i] = (unsigned) (i + t) * 2654435761u >> 8;
        uk_win_present ();                              /* the server shows it */
        kapi_pump_wait (16);                            /* the events (AppKit's pump) */
    }
    return 0;
}
```

The adaptive widgets (`uikit/adapt.h`, `sidepanel.h`, `form.h`, PocketUI's phase P6; docs/03 §5.10.6): the same binary runs on the desktop, in pocket and in console; the app says **what** its parts are and each UIKit shows them for its mode — on the desktop exactly as before. A navigation **SidePanel** (a rail, a drawer, a console column away from the desktop), tools with **priorities** (pocket: one row and »), **column roles** (portrait: cards), a **FormDialog** (portrait: a sheet), a field's **type**, the **size class**:

```
#include "uikit/uikit.h"
#include "uikit/toolbar.h"

SidePanel *nav = new SidePanel (0, 40, 200, 560, UK_SP_LEFT, UK_SP_NAVIGATION);
nav->addHeading ("LIBRARY");  nav->addItem (1, "Songs", WKG_HOME);  nav->setBadge (1, 12);
nav->onSelect = [] (SidePanel &, int id) { show (id); };
bar->setPriority (cut, UK_TB_IF_ROOM, 2);  bar->setPriority (settings, UK_TB_OVERFLOW);
grid->setColumnRole (0, UK_COL_PRIMARY);  grid->setColumnRole (1, UK_COL_SECONDARY);  grid-
>setColumnRole (3, UK_COL_DETAIL, 1);
uk_set_input_type (urlBox, UK_IN_URL);
void relayout () {                                      // the window's resize,
Root::onSizeClass, nav->onPresentation
    nav->place (0, 40, 200, height - 40);
    content->left = nav->reservedWidth ();              // 200 whole, 48 a rail, 0 a drawer
    if (uk_size_class () == UK_SC_NARROW) { /* portrait: the app's own choices */ }
}
FormDialog f ("New Playlist");  f.addRow ("Name:", new Textbox (0, 0, 240, 26, ""));
f.addButton ("Create", UK_FB_DEFAULT, 1);  f.addButton ("Cancel", UK_FB_CANCEL, 0);  int r =
f.run ();
```

5. SystemKit — talking to the system and the other programs

`#include "systemkit/systemkit.h"` — link `lib/systemkit.imp.a` (C++) or `lib/systemkit.imp_c.a` (C). The one header brings every subject below (a C program gets those written in C: notifications, volume, preload list, autostart, the applets' protocol).

A notification (a bubble under the menu bar; `notify_action` adds what a click starts):

```
#include "systemkit/systemkit.h"

notify ("Backup", "3 files copied");
notify_action ("Updates", "2 updates available", "control pkgman");
```

The clipboard, shared by every application:

```
#include "systemkit/systemkit.h"

clip_set_text ("copied text");

char buf[1024];
if (clip_get_text (buf, sizeof buf)) { /* paste buf */ }
```

```
int w, h;
unsigned *px = clip_get_image (&w, &h);              // a picture, if one was copied
if (px) { /* ... */ delete [] px; }
```

The trash — a file is moved there instead of being deleted:

```
#include "systemkit/systemkit.h"

if (trash_move ("SD:/docs/old.txt")) notify ("Trash", "old.txt moved to the trash");
int n = trash_count ();                              // what the trash holds
```

Opening a file with the right application (the user's file associations):

```
#include "systemkit/systemkit.h"

char app[32];
if (fa_app_for ("SD:/photos/cat.png", app, sizeof app)) { /* "imageview" */ }
fa_open ("SD:/photos/cat.png");                      // a folder, an app, a document: opened
```

The volume, the wallpaper, the dock — a settings program changes the file and tells the system:

```
#include "systemkit/systemkit.h"

volume_save (7, 0);                                  // 0..10, not muted: kept for the next
start

Wallpaper wp;
wp_load (wp);                                        // SD:/etc/wallpaper.ini
/* change wp ... */
wp_save (wp);

dock_reload ();                                      // the dock reads its settings again
```

Starting a program at every boot — a line of `SD:/etc/autostart` looked for, or added where it belongs (after a given line, else before `preload /boot`, which stays last; Setup's held-back `#setup:` lines respected), and the user told so:

```
#include "systemkit/systemkit.h"

int r = autostart_ensure ("run stickies", "run agenda", "# Stickies: the pinned notes on the
desktop");
/* 2 added, 1 already there (autostart_has), 0 not written */
```

The system's language (the Control Panel's Language & Region): an app's words go through UIKit's `TR ("...")` after `uk_lang_init ()` (section 3); a program with words of its own asks SystemKit:

```
const char *code = locale_language ();               // "en", "fr" (system.ini's Language=;
none: "en")
for (int i = 0; i < locale_language_count (); i++)   // the languages Onyx speaks, each in
its own words
    list->add (locale_language_name (i));
locale_set_language ("fr");                          // kept: the programs started next
speak it
locale_set_zone (0);                                 // a time zone (locale_zone_count /
_city / _offset): now, and kept
```

The time zones at an instant (the Clock's World tab): `locale_zone_offset (z)` judges the summer time by today's date only; `locale_zone_offset_at (z, utc_minutes)` gives a zone's offset at an exact instant,

the hour of the change counted (the EU's at 01:00 UTC, the US' at 02:00 local) — a city's time is its UTC plus that offset, right on the night of a change whatever the local day says:

```
struct kapi_clock_info ci;
if (kapi_clock_info (&ci) == 0 && (ci.flags & KAPI_CLOCK_REALTIME_VALID))
{
    long long utc = ci.utc_us / 60000000;                          // minutes since 1970, UTC
    int ny = 16;                                                   // locale_zone_city (16):
"New York"
    long long there = utc + locale_zone_offset_at (ny, utc);       // its wall minute now
}
```

The kernel reads `timezone=` once at boot; `locale_zone_sync ()` puts the system's clock on the zone chosen in Language & Region or Setup (`zone=` only — a card with a `timezone=` and no `zone=` is left as it is) when the summer time begins or ends: called once a minute (clockd, the Clock's service, does), on 25 October 2026 at 03:00 CEST the clock becomes 02:00 CET, `timezone=` rewritten (1 changed, 0 not).

The interface's mode and its session (`session.h`, 2026-10-08): the mode — `SESSION_DESKTOP`, `SESSION_POCKET`, `SESSION_CONSOLE` — is `system.ini`'s `shell=`; each mode's programs are its session file `SD:/etc/session/<mode>`. A settings page switches the mode as the Control Panel's Mode applet does: `/bin/session` is started, closes the open programs (each one asks about its unsaved work), and the caller polls it:

```
void *h = session_switch_start (SESSION_POCKET, 0, 0);     // the caller and its parents are
kept open, then ended
/* ... each turn: */ if (h && kapi_proc_done (h)) {
    int r = kapi_wait (h); h = 0;
    if (r == SESSION_WAITING_APPS) { char who[512]; session_waiting (who, sizeof who);   //
"name\ttitle" lines
        /* ask: Wait -> session_switch_start (m, SESSION_NO_ASK, 0); Force -> ...
SESSION_NO_ASK | SESSION_FORCE */ }
}
int m = session_mode ();                                       // the mode chosen (no line: the
desktop)
char names[512]; session_programs (m, names, sizeof names); // "menubar\nterminal\n": what
its session starts
```

`autostart_has` / `autostart_ensure` look in the autostart and in the session files: a line added after an anchor goes into the file that has it.

The pocket shell (`shell.h`, 2026-10-08): in the pocket mode a program may open one of the pocket shell's screens (`pocketshell` serves the IPC service `shell`); on the desktop nothing serves it and the call answers 0:

```
if (shell_running ()) shell_ask (SHELL_MSG_QUICK);          // quick settings and the
notifications (else: our own)
```

The documents opened last (`recent.h`, 2026-10-08): `fa_open` notes every file it opens with its app; an app that opens a file by itself (its own Open dialog) may note it too. The pocket launcher's Recent shows them:

```
recent_doc_add ("SD:/docs/letters-tour.rtf");              // first of SD:/etc/recent-docs (24
at most, a path once)
struct recent_doc d[12]; int n = recent_docs (d, 12);       // the latest first:
d[i].path, .date (YYYYMMDD), .time (HHMM)
```

The screen's resolution (`display.h`, 2026-10-09): the sizes offered, the one kept for the next start, and in the console mode each app's own (an emulator's `resolution =`, `SD:/etc/console.ini [screen]`). The Display applet and the console's Display page share it:

```
int w, h;
const char *what = display_mode (8, &w, &h);                // 1920 x 1080, "16:9, Full HD"
(English: translate it)
if (kapi_screen_set (w, h) == 0) display_save_size (w, h);  // applied, then kept in
SD:/cmdline.txt
display_game_set ("n64emu", 1024, 768);                     // the N64 emulator at 1024 x 768
while it plays
display_game_set ("gcemu", DISPLAY_SYSTEM, 0);              // ... the system's size;
DISPLAY_OWN: its app.txt's again
if (display_game_size ("snesemu", &w, &h)) { /* the size consolehome switches to */ }
display_save_option ("opaque", "0");                        // another kernel option of
SD:/cmdline.txt (0: removed)
char v[8]; if (display_saved_option ("opaque", v, sizeof v) > 0) { /* "0" */ }
```

The screen at the size of the app in front (`display.h` `display_follow`, 2026-10-26): what the console's home and the pocket shell both do -- an app's own size while it is in front (its `resolution =`, `console.ini`, what it asked with `display_game_ask`), the system's again at home. A shell keeps one `struct display_follow`:

```
static struct display_follow f;  display_follow_init (&f, "pocketshell");
display_follow_launch (&f, "cosmozoom");          // opening it: its size now, before it
reads the screen's
display_follow_ask (&f, front, w, h);             // SHELL_MSG_SCREEN received
display_follow_tick (&f, front, names, n);        // every 0.1 s: the app in front ("" at
home), the programs running
```

The keyboard and the mouse (`input.h`, 2026-10-09): the layouts on the card, the one in use, taking one (kept for every start), whether it is AZERTY or QWERTZ (a virtual keyboard follows it), the wheel's speed:

```
char maps[INPUT_KEYMAPS_MAX][12]; int n = input_keymaps (maps, INPUT_KEYMAPS_MAX);   // "BE",
"DE", "FR", "US"...
input_keymap_set ("BE");                                    // INPUT_KEPT: loaded now and in
SD:/etc/autostart
if (input_keymap_kind ("FR") == 1) { /* AZERTY */ }
input_wheel_save (3);                                       // SD:/etc/theme.txt wheelspeed=
(apply: uk_win_wheel_set)
```

Its part (a header of its own, beside `systemkit.h`)	Subject
`notify.h`	Notifications
`clipboard.h` (`clipproto.h`: its protocol)	The clipboard
`trash.h`	The trash
`fileassoc.h`	Which application opens which file
`volume.h`	The master volume, the output, each program's volume
`wallpaper.h`	The wallpaper's settings and its painter
`dockconf.h`	The dock's settings
`preloadini.h`	The programs loaded ahead at boot

Its part (a header of its own, beside `systemkit.h`)	Subject
`autostart.h`	The programs started at boot (`SD:/etc/autostart` and the session files)
`applet_proto.h`	A settings applet shown inside the Control Panel
`locale.h`	The system's language and time zone (`SD:/etc/system.ini`)
`session.h`	The interface's mode (desktop, pocket, console) and its session: `SD:/etc/session/<mode>`, the switch
`shell.h`	The pocket shell's screens asked (`shell_ask (SHELL_MSG_HOME / _SWITCHER / _QUICK / _SEARCH / _CLIPBOARD)`, `shell_running ()`): the menu bar's way in pocket
`recent.h`	The documents opened last (`recent_doc_add`, `recent_docs`: `SD:/etc/recent-docs`; `fa_open` notes them)
`display.h`	The screen's resolution: the sizes, the one kept (`SD:/cmdline.txt`), the console's games' own (`SD:/etc/console.ini [screen]`)
`input.h`	The keyboard's layouts (`SD:/etc/keymaps`, kept in `SD:/etc/autostart`) and the mouse wheel's speed

6. NetKit — the network

`#include "netkit/netkit.h"` — link `lib/netkit.imp.a` (C++) or `lib/netkit.imp_c.a` (C). The `HttpClient` class is apart: `#include "netkit/http.hpp"`.

A page fetched (HTTP/1.0, no allocation: the caller gives the buffer):

```
#include "appkit/appkit.h"
#include "netkit/netkit.h"

static char buf[32 * 1024];

int main (void)
{
    http_response r;
    if (http_get ("http://example.com/", buf, sizeof buf, &r) < 0 || !r.ok)
    {
        ax_putln ("request failed");
        return 1;
```

```
    }
    kapi_stdout_write (r.body, (unsigned) r.body_len);
    return 0;
}
```

A web API (HTTP/1.1, headers, HTTPS with the TLS transport) — the `HttpClient` class:

```
#include "netkit/http.hpp"

static char buf[64 * 1024];
HttpClient http;
http.user_agent ("MyApp/1.0").accept ("application/json");
HttpResponse r = http.get ("http://api.example.com/status", buf, sizeof buf);
if (!r.is_error ()) { /* r.status, the body in buf */ }
```

An FTP server as a volume: once logged in, `FTP:host/dir/file` is a path like any other, for every program.

```
#include "netkit/netkit.h"

if (ftpfs_login ("ftp.example.com", "me", "secret", 1))      // 1: remembered
{
    void *f = kapi_open ("FTP:ftp.example.com/pub/readme.txt");
    /* kapi_read ... kapi_close */
}
```

The known Wi-Fi networks (`wifi.h`, 2026-10-09): `SD:/etc/wpa_supplicant.conf` read and written with every network kept (the one joined last first), joining one at once (the scan itself is AppKit's `kapi_wlan_scan`). The menu bar's Wi-Fi menu, the Wi-Fi applet and the console's Wi-Fi page share it:

```
struct wifi_known k[WIFI_KNOWN_MAX]; char cc[8];
int n = wifi_known_load (k, WIFI_KNOWN_MAX, cc, sizeof cc);   // the networks, the country
int r = wifi_join ("Maison", WLAN_SEC_WPA2, "the password");  // WIFI_OK (joining),
WIFI_SAVED (a reboot joins it), WIFI_E*
wifi_forget ("Voisin");
wifi_set_country ("BE");
```

The passwords are in clear text on the card (the radio needs them).

The raw sockets (`kapi_tcp_connect`, `kapi_tcp_send`…) are AppKit's; NetKit is where protocols built on them belong.

7. FileKit — files, folders, archives

`#include "filekit/filekit.h"` — link `lib/filekit.imp.a`.

A whole file, read and written:

```
#include "filekit/filekit.h"

void *data; unsigned n;
if (fk_load ("SD:/docs/notes.txt", &data, &n) == 0)
{
    /* data: n bytes, a 0 after them */
    fk_save ("SD:/docs/notes.bak", data, n);
    fk_free (data);
}
```

Folders and paths:

```
#include "filekit/filekit.h"

char path[FS_PATHL];
fs_join (path, sizeof path, "SD:/docs", "report.txt");
if (!fs_exists (path)) { /* ... */ }

fk_mkdirs ("SD:/backup/2026/docs");                    // every folder of the path
fk_copy ("SD:/docs", "SD:/backup/2026/docs", 0, 0);    // a file, or a folder and all it holds
fk_remove ("SD:/tmp/work");                            // the same, removed
```

A ZIP archive, read:

```
char err[80];
fk_zip *z = fk_zip_open ("SD:/downloads/pack.zip", err, sizeof err);
if (z)
{
    struct fk_zip_entry e;
    for (int i = 0; i < fk_zip_count (z); i++)
        if (fk_zip_entry (z, i, &e)) ax_putln (e.name);
    fk_zip_extract_all (z, 0, "SD:/downloads/pack", 0, 0);
    fk_zip_close (z);
}
```

A ZIP archive, written:

```
fk_zipw *w = fk_zipw_create ("SD:/backup/docs.zip");
fk_zipw_add (w, "SD:/docs", 0);                        // the folder and all it holds
fk_zipw_close (w, 0, 0, err, sizeof err);
```

Other archive formats (tar, tar.gz, gzip) are read through the same calls (`fk_arc_*`); a program asks the library which formats it handles (`fk_arc_formats`) instead of keeping its own list.

A settings or progress file with sections, read and written back (`filekit/kvtext.h`, C and C++): the entries stay in the file's order, the keys a program does not know are kept, values may hold new lines.

```
fk_kv *kv = fk_kv_load ("SD:/apps/game.app/progress.ini", FK_KV_ESCAPES);
if (!kv) kv = fk_kv_new (FK_KV_ESCAPES);                    // no file yet: a fresh start
int stars = atoi (fk_kv_get (kv, "Player", "level1", "0"));
fk_kv_set (kv, "Player", "level1", "3");
fk_kv_set (kv, "", "last", "level1");                       // "": before the first [header]
fk_kv_save (kv, "SD:/apps/game.app/progress.ini", "# my game's progress");
fk_kv_free (kv);
```

A level pack (`FK_KV_PIPES`): many `[level]` blocks (`fk_kv_block`, `fk_kv_blocks`, `fk_kv_block_name`), a value going on over the `|` lines that follow, and the line of each value (`fk_kv_line`) for an error message. Circuits reads its packs and its `progress.ini` so.

Blocks of the same name written (the Clock's `[alarm]` list; `fk_kv_set` reaches only a name's first block): a new block made at the end, its keys set and read by its number —

```
fk_kv *kv = fk_kv_new (0);
int b = fk_kv_block_new (kv, "alarm");                      // its number (1-based), -1 no memory
fk_kv_block_set (kv, b, "time", "07:00");                   // replaced in block b, else added at its end
```

```
const char *t = fk_kv_block_get (kv, b, "time", "");        // block b's value ("" if none)
fk_kv_save (kv, "SD:/apps/clock.app/alarms.txt", "# Clock -- the alarms");
```

8. ImageKit — pictures

`#include "imagekit/imagekit.h"` — link `lib/imagekit.imp.a`.

ImageKit is the system's one picture codec: BMP, GIF, PNG, JPEG, PCX and WebP read; PNG, JPEG, BMP and GIF written.

A picture read, turned, saved in another format:

```
#include "imagekit/imagekit.h"

ik_image *im = ik_load ("SD:/photos/cat.jpg", 0);
if (im)
{
    int w = ik_width (im), h = ik_height (im);
    unsigned *px = ik_pixels (im);                    // w * h pixels, 0xAARRGGBB
    ik_rotate (im, 1);                                // a quarter turn clockwise
    ik_save (im, "SD:/photos/cat.png", 90);           // the format: the extension's
    ik_image_free (im);
}
```

Pixels of the program's own, saved:

```
ik_image *shot = ik_image_from (pixels, width, height, width);
ik_save (shot, "SD:/screenshots/shot.png", 0);
ik_image_free (shot);
```

The formats, asked — a file chooser's filter follows what the library knows:

```
struct ik_format f[16];
int n = ik_formats (f, 16);
```

Resizing (`ik_scale`), flipping (`ik_flip`), the photo adjustments and the EXIF orientation are in the same header.

9. AudioKit — sound

`#include "audiokit/audiokit.h"` — link `lib/audiokit.imp.a`.

Several programs play at once: the system mixes them, each with its own volume (Control Panel ▸ Sound).

A sound file (WAV, MP3, OGG, FLAC, MIDI, FM Tracker songs), played in the background:

```
#include "audiokit/audiokit.h"

if (ak_play ("SD:/music/song.mp3", 0) < 0) ax_putln (ak_play_error ());
ak_play_volume (80);                                  // 0..100
/* ... the program goes on ... */
ak_play_stop ();
```

Notes (the General MIDI instruments, a SoundFont):

```
ak_note_on (0, 60, 100);                              // channel, key (60 = middle C),
velocity
```

```
kapi_msleep (400);
ak_note_off (0, 60);
```

A sound effect (the FM synthesizer's voices — what the games use):

```
ak_fm_start (0, 440000, 0, 200);                    // voice 0, 440 Hz (in milli-Hz),
volume 0..255
kapi_msleep (120);
ak_fm_stop (0);
```

The program's own samples (an emulator, a synthesizer): stereo, 16 bits, 44.1 kHz (`AUDIOKIT_RATE`).

```
if (ak_out_open (256, 1024) == 1)
{
    short frames[256 * 2];
    /* fill frames ... */
    ak_out_write (frames, 256);                     // waits for room
    ak_out_close ();
}
```

The microphone (the one chosen in Control Panel ▸ Sound: USB or I2S): mono, 16 bits, at the rate you ask. One program records at a time.

```
int rate = ak_in_open (16000);                      // 0: another program records, -1: no
microphone
if (rate > 0)
{
    short s[1600];
    int n = ak_in_read (s, 1600);                   // never waits: 0 = nothing yet, -1 =
gone
    int level = ak_in_peak (s, n);                  // 0..32767
    /* ... ak_wav_begin / ak_wav_write to keep it ... */
    ak_in_close ();
}
```

10. FontKit — fonts and text

`#include "fontkit/uikitface.h"` or `"fontkit/fonts.h"` — link `lib/fontkit.imp.a`.

FontKit is FreeType for the applications, with the card's fonts (`SD:/res/fonts`).

Anti-aliased text in a UIKit application — one call before the widgets are built:

```
#include "uikit/uikit.h"
#include "fontkit/uikitface.h"
using namespace uikit;

int main (void)
{
    ft_uikit_install ("DejaVu Sans", 13);           // every widget's text from now on
    Root root (400, 200, "Smooth text");
    root.addChild (new Label (12, 12, 376, 24, "FreeType draws this"));
    root.run ();
    return 0;
}
```

The font manager — the families installed, a font at a size, text measured (a word processor, a canvas):

```
#include "fontkit/fonts.h"

fnt::init ();
for (int i = 0; i < fnt::count (); i++) ax_putln (fnt::name (i));

int fam = fnt::find ("DejaVu Serif");
fnt::Font *f = fnt::get (fam, 0, 14 * 64);            // style 0 (regular), 14 px in 26.6
int width = fnt::advance (f, 'A');                    // in 26.6 units
```

11. PrinterKit — printing

`#include "printerkit/printerkit.h"` — link `lib/printerkit.imp.a`.

A program prints in four steps: the Print dialog, a job, its pages, the end. Lengths are points (1/72 inch) from the page's top-left corner; colours are `0xRRGGBB`. The pages are recorded: the print service turns them into what the printer takes — or into a PDF file.

```
#include "printerkit/printerkit.h"

PrintSetup s;
print_setup_default (&s);
PrintDialogInfo di = { sizeof di, "My document", 1, 0, 0, 0, 0 };
if (print_dialog (&s, &di))                           // the printer, the paper, the pages
{
    PrintJob *j = print_begin (&s, "My document");
    int font = print_font (j, "DejaVu Sans", 0);
    if (print_page (j, 0, 0))                         // 0, 0: the paper chosen
    {
        print_text (j, font, 12, 72, 84, "Hello, printer", 0x000000);
        print_line (j, 72, 90, s.paper_w - 72, 90, 0.75f, 0x808080);
    }
    print_end (j);                                    // to the print queue
}
```

Rectangles, paths (lines and curves, filled or stroked) and pictures are drawn the same way (`print_rect`, `print_path_*`, `print_image`).

12. GPIOKit — the 40-pin header

`#include "gpiokit/gpiokit.h"` — link `lib/gpiokit.imp.a` (C++) or `lib/gpiokit.imp_c.a` (C).

The Raspberry Pi's header from a program: a pin's mode, its level, PWM, edges, the I2C bus and SPI. Pins are **BCM GPIO numbers** (GPIO 17 is the header's pin 11: `gk_header_pin`). **The levels are 3.3 V**: never wire 5 V to a pin; at most about 16 mA a pin (an LED through a 330 Ω resistor). A pin is the program's from its first use until it gives it back (`gk_mode (pin, GK_FREE)`, `gk_release`) or ends — the system then makes it an input again, after a crash too. The system's pins (GPIO 14 / 15, the serial console; 0 / 1, the HAT EEPROM) and a pin another program has are refused: every call returns ≥ 0, or a negative `GK_E*` that `gk_error` says in words.

GPIOKit is the one kit besides AppKit that calls the kernel itself (kapi v92 `gpio_ctl`; the user's exception, docs/03 §5.10); it is shipped with the kernel in the package `onyx`.

An LED, a button (C):

```
#include "gpiokit/gpiokit.h"
```

```
gk_mode (17, GK_OUT);                                   // the LED: GPIO 17 -> 330 ohm -> LED
-> GND
gk_mode (27, GK_IN_PULLUP);                             // the button: GPIO 27 -> button -> GND
(reads 0 pressed)
gk_edges (27, GK_FALLING);                              // its presses queued, with their time
for (;;)
{
    gk_event e[8];
    int n = gk_events (e, 8, 1000);                     // waits up to 1 s for the first
    for (int i = 0; i < n; i++) gk_toggle (17);         // each press turns the LED over
}
```

PWM and a servo (GPIO 12, 13, 18, 19; 12 / 18 and 13 / 19 share a channel):

```
gk_pwm (18, 1000, 2500);                                // 1 kHz, 25 % high (duty in 1/10000)
gk_servo (18, 1500);                                    // a servo: 50 Hz, a 1.5 ms pulse (the
middle)
```

I2C (GPIO 2 SDA, GPIO 3 SCL) — a BME280's id, a scan:

```
if (gk_i2c_open (0) == 0)                               // 0: 100 kHz
{
    unsigned char map[16];
    int n = gk_i2c_scan (map);                          // bit a of map: something answers at a
    if (gk_i2c_reg_read (0x76, 0xD0) == 0x60) ax_putln ("a BME280 at 0x76");
    gk_i2c_close ();
}
```

SPI 0 (GPIO 8 CE0, 7 CE1, 9 MISO, 10 MOSI, 11 SCLK): `gk_spi_open (1000000, 0)`, then `gk_spi_transfer (0, tx, rx, n)`.

C++ has small classes that give the pin back by themselves:

```
gpiokit::Pin led (17, GK_OUT);
if (!led.ok ()) ax_putln (gk_error (led.status ()));
led.write (1);
gpiokit::I2CDevice bme (0x76);
int id = bme.reg (0xD0);
```

No hardware? GPIOKit has a **simulator** — a board in memory: the inputs set by `gk_sim_input`, the PWM's waves, an I2C bus with an SSD1306 display (0x3C: `gk_sim_display` gives its pixels) and a BME280 sensor (0x76), SPI looped back. `gk_sim (1)` switches to it; it runs by itself where the system has no GPIO, and on a PC. GPIO Lab (docs/04) is built on GPIOKit; BASIC has statements for it (docs/04 §13 *GPIO*).

13. GameKit — the games of the card's ROMs

`#include "gamekit/gamekit.h"` — link `lib/gamekit.imp.a` (C++) or `lib/gamekit.imp_c.a` (C).

What the Game Library and the console's home share about the games: **the consoles** the installed emulators play (each emulator's `app.txt`: `games = Game Boy Color: gbc; Game Boy: gb`, `order = 50` — an emulator's package installed, its console appears), **the watched folders** (`SD:/apps/gamelib.app/config.ini`, one `folder =` line each; `SD:/roms` when none is said), **the ROMs** found in them and their sub-folders by their extension — sorted by console, then by name, each with a name to show (`ZeldaOracleOfSeason.gbc` → *Zelda Oracle Of Season*) — and **their pictures**, the title screens the Game Library makes (`SD:/apps/gamelib.app/thumbs/<file>.thm`, 160 × 144). The gamepads' buttons and their settings are not in it: `Include/gamepad.h` keeps them.

Every Game Boy Advance game, its picture (C):

```c
#include "gamekit/gamekit.h"

static struct game_system sys[GAMES_SYSTEMS_MAX];
static char dirs[GAMES_FOLDERS_MAX][GAMES_PATH];
static struct game g[256];
static unsigned pic[GAMES_THUMB_W * GAMES_THUMB_H];

int ns = games_systems (sys, GAMES_SYSTEMS_MAX);        // the consoles, by their order
int nd = games_folders (dirs, GAMES_FOLDERS_MAX);       // the watched folders
int n = games_scan (dirs, nd, sys, ns, g, 256);         // by console, then by name
for (int i = 0; i < n; i++)
    if (ax_streq (sys[g[i].sys].name, "Game Boy Advance"))
    {
        ax_putln (g[i].name);                           // "Star Courier"
        if (games_thumb_load (&g[i], pic)) { /* 0xRRGGBB, 160 x 144 */ }
    }
```

A game is opened with its emulator by AppKit (`lx_open (g[i].path, "")`: the emulator found by the same `app.txt`). A folder added: `games_folders_save (dirs, nd + 1)`. A program that makes the pictures saves them with `games_thumb_save`.

14. InputKit — the keyboard and the gamepads as one input

`#include "inputkit/inputkit.h"` — link `lib/inputkit.imp.a` (C++) or `lib/inputkit.imp_c.a` (C). Reference: `docs/21-INPUTKIT.md`.

A program asks for an **action** — `IN_UP`, `IN_ACCEPT`, `IN_NEXT`... — never for "the arrow key or the D-pad". Each action has its **sources**, keys (with their modifiers) and pad buttons, in one file, **`SD:/etc/input.ini`** (two sections, `[keys]` and `[pad]`; without the file, or for what it does not say, the defaults):

action	keys	pad	what it is for
`up` `down` `left` `right`	the arrows	the D-pad (and the left stick)	the focus, the caret
`accept`	Enter	A	open, confirm, activate
`back`	Esc	B	cancel, close, back
`alt1`	Delete	X	the second action: close the chosen app, remove
`alt2`	Shift+F10	Y	the options: a context menu
`prev` / `next`	Ctrl+Page Up, Ctrl+Shift+Tab / Ctrl+Page Down, Ctrl+Tab	L / R	the tab, the category, the section before / after
`menu`	F10	—	the app's menus (PocketUI's CMD key is the server's

action	keys	pad	what it is for
			own)
`home`	Ctrl+Esc	the middle button	Home
`start` `select` `l2` `r2`	—	Start, Select, L2, R2	a game's

Which raw button of a given pad model is "A" stays **`gamepad.h`**'s business (`SD:/etc/gamepad.ini`, the Control Panel's Gamepad applet): InputKit is built on it, and the emulators that still read `gamepad.h` are untouched.

Two ways to read, for two kinds of programs:

- **events**, for an interface — `in_poll` gives the pads' actions *pressed*, *repeated* (the directions, at the keyboard's cadence) and *released*, each saying its source (so a hint shows the right glyph: `in_binding`); a key the program received becomes its action with `in_key_action (key, kapi_get_modifiers ())`. Only the program that has the keys gets a pad's events: a background app never moves;
- **state**, for a game — `in_state (player, &st)`: the actions held now (`st.held & IN_BIT (IN_ACCEPT)`), the sticks, per player; player 0 also reads the keyboard's held keys. A program that reads the state **owns the pad** (`in_claimed`): the interface's navigation leaves it alone.

```
#include "inputkit/inputkit.h"

// an interface with its own loop: the pad and the keys through the same switch
static void act (int action)
{
    switch (action)
    {
    case IN_LEFT:   move (-1); break;
    case IN_RIGHT:  move (1); break;
    case IN_ACCEPT: open_chosen (); break;
    case IN_BACK:   close_me (); break;
    case IN_NEXT:   next_page (); break;
    }
}
static void on_key (unsigned long, int, long key) { int a = in_key_action ((int) key,
kapi_get_modifiers ()); if (a >= 0) act (a); }
// ... each turn:
struct in_event ev[8];
for (int i = 0, n = in_poll (ev, 8); i < n; i++) if (ev[i].kind != IN_RELEASE) act
(ev[i].action);

// a game: the state
struct in_state st;
if (in_state (0, &st)) { if (st.held & IN_BIT (IN_LEFT)) x--; if (st.held & IN_BIT
(IN_ACCEPT)) jump (); }
```

A UIKit app writes none of this: a `Root` takes its navigation from InputKit (docs/03 §5.10.8) — the pad's directions, A, B and X act as the arrows, Enter, Esc and Delete on the focused control, L / R step the window's tabs, Y is the context menu. Text is not InputKit's: characters keep coming as keys.

15. Adding to a kit, creating a kit

Before writing a helper, look for it in the kits. If two programs could use it, it belongs to a kit.

Adding a function to a kit

1. **Declare** it in the kit's header, with a comment that says what it does.
2. **Write its code** in the kit's source (for SystemKit, NetKit and `filekit/fsutil.h`: the `.inc` beside the header).
3. **Build**: the kit's table (`<kit>.abi`) gets the new name at its end. Commit the `.abi`. A name is never removed nor renamed: a function that is no longer needed keeps a body.
4. **Raise the package's version** (`tools/pkg/versions.ini`: `1.<the table's size>.0`) and what the applications need (`needs = <kit> >= 1.<size>` in `tools/pkg/packages.ini`).
5. **Document** it: its comment in the header is its documentation — run `python tools/docgen/kitdocs.py` (the kit's reference follows), add an example here if it deserves one, then `python docs/build_docs.py`.

Creating a kit

When a domain has no kit (SystemKit and NetKit are the models — small, with no dependency):

- a folder `user/Kits/<kit>/`: **one header a program includes, `<kit>/<kit>.h`** — it may bring a header per subject, each declaring its functions (`XK_API int f (...);`) with a `.inc` beside it for the code — and one source `<kit>.cpp` that compiles the code into the library;
- its rules in `user/Makefile` (copy SystemKit's block): the library `lib/<kit>.so`, its import archives `lib/<kit>.imp.a` (C++) and `lib/<kit>.imp_c.a` (C);
- its package in `tools/pkg/packages.ini` (`required = 1`, `files = lib/<kit>.so lib/<kit>.bi` — the `.bi` is what BASIC reads of it, made by the build: *From BASIC* above).

The header of a kit a C program may use is written in C.

16. Quick reference

I want to…	Kit	Call
read or write a file	AppKit	`kapi_open`, `kapi_read`, `kapi_save_file`
load a whole file in memory	FileKit	`fk_load`, `fk_save`
copy or remove a folder	FileKit	`fk_copy`, `fk_remove`
build a path	FileKit	`fs_join`, `fk_path_join`
read or write a ZIP	FileKit	`fk_zip_open`, `fk_zipw_create`
read and write a settings / progress file with sections	FileKit	`fk_kv_load`, `fk_kv_set`, `fk_kv_save`
print to the console	AppKit	`ax_puts`, `ax_putln`
read an `.ini` file	AppKit	`app_ini_load`, `app_ini_get`
start another program	AppKit	`lx_launch`, `lx_open`
open a window with widgets	UIKit	`Root`, `Label`, `Button`…
open a bare window,	UIKit	`uk_win_create`,

I want to…	Kit	Call
draw its pixels		`uk_win_present`, `uk_win_on_key`
take the whole screen (a game, an emulator)	UIKit (then AppKit's `kapi_present_fb`)	`uk_win_fullscreen_begin`, `uk_win_fullscreen_end`
list, raise, move the windows (a shell)	UIKit	`uk_win_list`, `uk_win_raise`, `uk_win_place`
ask the user (message, file, colour)	UIKit	`uk_messagebox`, `uk_file_open`, `uk_color_dialog`
lay the app out for every mode (desktop, pocket, console)	UIKit	`SidePanel`, `ToolBar::setPriority`, `DataGrid::setColumnRole`, `FormDialog`, `uk_size_class`, `Root::onSizeClass`
load an icon	UIKit	`ui::icon_load`
list the card's games (ROMs), their consoles, their pictures	GameKit	`games_systems`, `games_folders`, `games_scan`, `games_thumb_load`
show a notification	SystemKit	`notify`, `notify_action`
copy / paste	SystemKit	`clip_set_text`, `clip_get_text`
know the system's language	SystemKit (UIKit's `TR ()` for the words)	`locale_language`, `uk_lang_init`
move a file to the trash	SystemKit	`trash_move`
open a file with its application	SystemKit	`fa_open`
fetch a web page	NetKit	`http_get`, `HttpClient`
reach an FTP server	NetKit	`ftpfs_login`, then `FTP:` paths
load or save a picture	ImageKit	`ik_load`, `ik_save`
resize or turn a picture	ImageKit	`ik_scale`, `ik_rotate`
play a sound file	AudioKit	`ak_play`
play a note or an effect	AudioKit	`ak_note_on`, `ak_fm_start`
send samples to the output	AudioKit	`ak_out_open`, `ak_out_write`
record the microphone	AudioKit	`ak_in_open`, `ak_in_read`
draw smooth text	FontKit	`ft_uikit_install`, `fnt::get`
print	PrinterKit	`print_dialog`,

I want to…	Kit	Call
		`print_begin`, `print_page`, `print_end`
light an LED, read a button	GPIOKit	`gk_mode`, `gk_write`, `gk_read`
wait for a button's press	GPIOKit	`gk_edges`, `gk_events`
dim an LED, move a servo	GPIOKit	`gk_pwm`, `gk_servo`
talk to an I2C / SPI device	GPIOKit	`gk_i2c_reg_read`, `gk_i2c_write_read`, `gk_spi_transfer`